

Les Objets en JAVASCRIPT

Introduction : Qu'est-ce qu'un objet ?

En JavaScript, un objet est une structure de données qui permet de regrouper des informations liées entre elles. C'est comme une boîte qui contient plusieurs compartiments, chacun ayant un nom (appelé propriété) et une valeur.

Imaginez que vous voulez représenter une personne en JavaScript. Une personne a un nom, un âge, une adresse... Plutôt que de créer une variable séparée pour chaque information, vous pouvez tout regrouper dans un seul objet !

1. Créer un objet : La syntaxe de base

1.1. Syntaxe littérale (la plus simple)

C'est la méthode la plus courante et la plus simple pour créer un objet :

```
// Création d'un objet représentant une personne
let personne = {
  nom: "Dupont",
  prenom: "Jean",
  age: 25,
  ville: "Paris"
};
```

Explication détaillée :

- Les accolades { } délimitent l'objet
- Chaque ligne à l'intérieur est une propriété (ou clé) avec sa valeur
- Le format est : nomPropriete: valeur
- Chaque propriété est séparée par une virgule ,

1.2. Les valeurs dans un objet

Un objet peut contenir différents types de valeurs :

```
let voiture = {
  marque: "Renault",           // Chaîne de caractères
  annee: 2020,                 // Nombre
  electrique: false,          // Booléen
  couleurs: ["rouge", "bleu"], // Tableau
  proprietaire: {              // Objet imbriqué
    nom: "Martin",
    age: 35 } };
```

2. Accéder aux propriétés d'un objet

2.1. Notation pointée (recommandée)

C'est la méthode la plus lisible et la plus utilisée :

```
let personne = {  
  nom: "Dupont",  
  age: 25  
};  
  
// Accéder aux propriétés  
console.log(personne.nom); // Affiche : "Dupont"  
console.log(personne.age); // Affiche : 25
```

2.2. Notation avec crochets

Utile quand le nom de la propriété contient des espaces ou est dans une variable :

```
let personne = {  
  nom: "Dupont",  
  "date de naissance": "15/03/1998"  
};  
  
// Avec crochets  
console.log(personne["nom"]); // "Dupont"  
console.log(personne["date de naissance"]); // "15/03/1998"  
  
// Avec une variable  
let propriete = "age";  
console.log(personne[propriete]); // Accède à personne.age
```

3. Modifier et ajouter des propriétés

3.1. Modifier une propriété existante

```
let personne = {  
  nom: "Dupont",  
  age: 25  
};  
  
// Modifier l'âge  
personne.age = 26;  
  
console.log(personne.age); // Affiche : 26
```

3.2. Ajouter une nouvelle propriété

```
let personne = {  
  nom: "Dupont",  
  age: 25  
};  
  
// Ajouter une nouvelle propriété  
personne.ville = "Lyon";  
personne.profession = "Développeur";  
  
console.log(personne);  
// Affiche : { nom: "Dupont", age: 25, ville: "Lyon", profession:  
"Développeur" }
```

3.3. Supprimer une propriété

```
let personne = {  
  nom: "Dupont",  
  age: 25,  
  ville: "Paris"  
};  
  
// Supprimer la propriété ville  
delete personne.ville;  
  
console.log(personne); // { nom: "Dupont", age: 25 }
```

4. Les méthodes : des fonctions dans les objets

Une méthode est une fonction qui appartient à un objet. Elle permet à l'objet d'effectuer des actions.

4.1. Créer une méthode

```
let personne = {  
  nom: "Dupont",  
  prenom: "Jean",  
  age: 25,  
  
  // Méthode pour se présenter  
  sePresenter: function() {  
    console.log("Bonjour, je m'appelle " + this.prenom + " " +  
this.nom);  
  }  
};  
  
// Appeler la méthode  
personne.sePresenter(); // Affiche : "Bonjour, je m'appelle Jean Dupont"
```

4.2. Le mot-clé 'this'

this fait référence à l'objet lui-même. Il permet d'accéder aux propriétés de l'objet depuis ses méthodes.

```
let compteBancaire = {  
  solde: 1000,  
  
  deposer: function(montant) {  
    this.solde = this.solde + montant;  
    console.log("Nouveau solde : " + this.solde + "€");  
  },  
  
  retirer: function(montant) {  
    if (montant <= this.solde) {  
      this.solde = this.solde - montant;  
      console.log("Nouveau solde : " + this.solde + "€");  
    } else {  
      console.log("Solde insuffisant !");  
    }  
  }  
};  
  
compteBancaire.deposer(500);    // Nouveau solde : 1500€  
compteBancaire.retirer(200);    // Nouveau solde : 1300€  
compteBancaire.retirer(2000);   // Solde insuffisant !
```

4.3. Syntaxe moderne des méthodes (ES6)

Depuis ES6, vous pouvez écrire les méthodes de manière plus courte :

```
let personne = {  
  nom: "Dupont",  
  prenom: "Jean",  
  
  // Syntaxe moderne (sans le mot 'function')  
  sePresenter() {  
    console.log(`Bonjour, je m'appelle ${this.prenom} ${this.nom}`);  
  },  
  
  direAge() {  
    console.log(`J'ai ${this.age} ans`);  
  }  
};
```

5. Parcourir un objet

5.1. Boucle for...in

Pour parcourir toutes les propriétés d'un objet :

```
let voiture = {  
  marque: "Peugeot",  
  modele: "308",  
  annee: 2021,  
  couleur: "blanc"  
};  
  
// Parcourir toutes les propriétés  
for (let propriete in voiture) {  
  console.log(propriete + " : " + voiture[propriete]);  
}  
  
// Affiche :  
// marque : Peugeot  
// modele : 308  
// annee : 2021  
// couleur : blanc
```

5.2. Object.keys(), Object.values(), Object.entries()

JavaScript fournit des méthodes pratiques pour travailler avec les objets :

```
let personne = {  
  nom: "Dupont",  
  prenom: "Marie",  
  age: 30  
};  
  
// Récupérer toutes les clés (noms des propriétés)  
let cles = Object.keys(personne);  
console.log(cles); // ["nom", "prenom", "age"]  
  
// Récupérer toutes les valeurs  
let valeurs = Object.values(personne);  
console.log(valeurs); // ["Dupont", "Marie", 30]  
  
// Récupérer les paires clé-valeur  
let entrees = Object.entries(personne);  
console.log(entrees);  
// [["nom", "Dupont"], ["prenom", "Marie"], ["age", 30]]
```

6. Vérifier l'existence d'une propriété

6.1. L'opérateur 'in'

```
let personne = {  
  nom: "Dupont",  
  age: 25  
};  
  
// Vérifier si une propriété existe  
if ("nom" in personne) {  
  console.log("La propriété 'nom' existe");  
}  
  
if ("ville" in personne) {  
  console.log("La propriété 'ville' existe");  
} else {  
  console.log("La propriété 'ville' n'existe pas");  
}
```

6.2. La méthode hasOwnProperty()

```
let personne = {  
  nom: "Dupont",  
  age: 25  
};  
  
if (personne.hasOwnProperty("nom")) {  
  console.log("La propriété 'nom' appartient à cet objet");  
}
```

7. Copier et fusionner des objets

7.1. Object.assign()

Copie les propriétés d'un ou plusieurs objets dans un objet cible :

```
let personne = {  
  nom: "Dupont",  
  age: 25  
};  
  
let adresse = {  
  ville: "Lyon",  
  codePostal: "69000"  
};  
  
// Fusionner les deux objets  
let personnComplete = Object.assign({}, personne, adresse);  
  
console.log(personnComplete);  
// { nom: "Dupont", age: 25, ville: "Lyon", codePostal: "69000" }
```

7.2. Opérateur de décomposition (Spread operator)

Syntaxe moderne et plus lisible pour copier/fusionner des objets :

```
let personne = {  
  nom: "Dupont",  
  age: 25  
};  
  
let adresse = {  
  ville: "Lyon",  
  codePostal: "69000"  
};  
  
// Fusion avec le spread operator (...)  
let personnComplete = { ...personne, ...adresse };  
  
console.log(personnComplete);  
// { nom: "Dupont", age: 25, ville: "Lyon", codePostal: "69000" }  
  
// Copier un objet  
let copie = { ...personne };  
console.log(copie); // { nom: "Dupont", age: 25 }
```


8. Exercices pratiques

Exercice 1 : Créer un objet produit

Créez un objet représentant un produit avec les propriétés suivantes : nom, prix, quantité, description. Affichez ensuite toutes les propriétés.

Exercice 2: Gestion d'un étudiant

Créez un objet étudiant avec : nom, prenom, notes (un tableau), et une méthode calculerMoyenne() qui calcule la moyenne des notes.

Exercice 3: Panier d'achat

Créez un objet panier avec une propriété articles (tableau vide) et les méthodes : ajouterArticle(article), retirerArticle(index), afficherPanier().

9. Points clés à retenir

1. Un objet regroupe des données liées sous forme de propriétés (clé : valeur)
2. On crée un objet avec des accolades { }
3. On accède aux propriétés avec la notation pointée objet.propriete ou les crochets objet['propriete']
4. Les méthodes sont des fonctions appartenant à un objet
5. this permet d'accéder aux propriétés de l'objet depuis ses méthodes
6. On peut ajouter, modifier ou supprimer des propriétés à tout moment
7. Utilisez Object.keys(), Object.values(), Object.entries() pour manipuler les objets

10.1. Système de gestion de bibliothèque

```
let bibliotheque = {
  nom: "Bibliothèque Municipale",
  livres: [
    {
      titre: "Le Petit Prince",
      auteur: "Antoine de Saint-Exupéry",
      annee: 1943,
      disponible: true
    },
    {
      titre: "1984",
      auteur: "George Orwell",
      annee: 1949,
      disponible: false
    }
  ],

  ajouterLivre: function(livre) {
    this.livres.push(livre);
    console.log(`Livres "${livre.titre}" ajouté à la bibliothèque`);
  },

  rechercherLivre: function(titre) {
    for (let livre of this.livres) {
      if (livre.titre === titre) {
        return livre;
      }
    }
    return null;
  },

  afficherLivresDisponibles: function() {
    console.log("Livres disponibles :");
    for (let livre of this.livres) {
      if (livre.disponible) {
        console.log(`- ${livre.titre} par ${livre.auteur}`);
      }
    }
  }
};

// Utilisation
bibliotheque.afficherLivresDisponibles();
let livre = bibliotheque.rechercherLivre("1984");
console.log(livre);
```

Conclusion

Les objets sont au cœur de JavaScript et vous les utiliserez quotidiennement dans vos projets. Ils permettent d'organiser votre code de manière logique et structurée.

Prenez le temps de bien comprendre les concepts de base : création d'objets, accès aux propriétés, méthodes et l'utilisation de this. Une fois ces fondamentaux maîtrisés, vous pourrez créer des applications complexes et bien organisées.

N'hésitez pas à pratiquer avec les exercices proposés et à créer vos propres objets pour modéliser des situations réelles !